

MEMORY ARCHITECT KIT

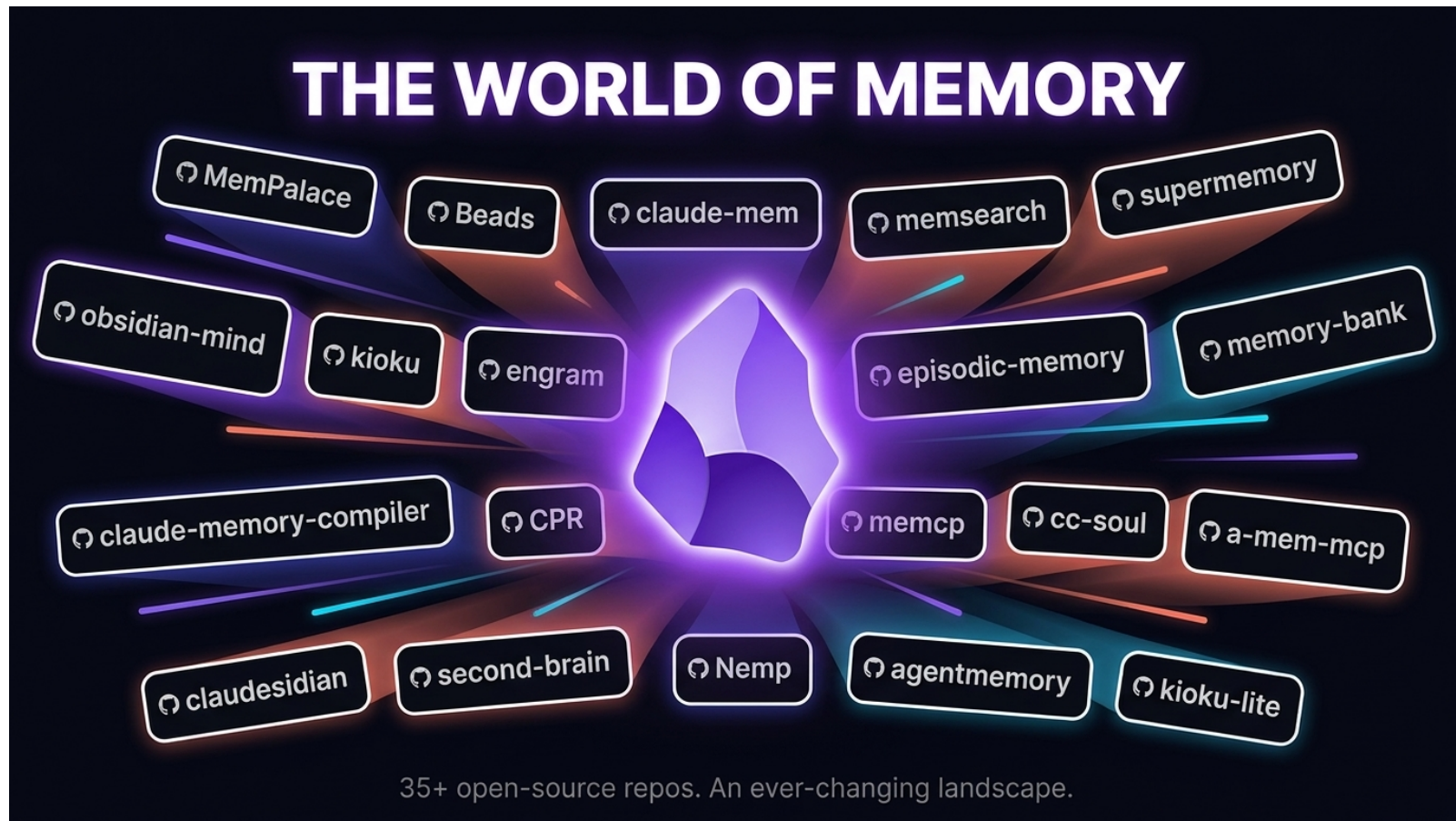
Diagram Guide

16 diagrams explaining how to design your perfect memory system.

by Mark Kashef | Early AI Dopters | <https://www.skool.com/earlyaidopters/about>

The World of Memory

35+ open source memory systems for Claude Code



- This is the landscape. MemPalace, Beads, memsearch, claude-mem, obsidian-mind, and dozens more.
- Some use vector databases, some use markdown files, some use knowledge graphs, some use Obsidian.
- Every single one says 'this is the best approach.' The question is: how do you choose?
- The answer: you don't pick one. You cherry-pick the best patterns from all of them.

Memory is a Fingerprint

Same word. Three completely different architectures.

MEMORY IS A FINGERPRINT



Same word. Three different architectures.

- A high-volume e-commerce business needs memory that tracks customer patterns, seasonal trends, campaign performance.
- A wealth manager needs memory that carries forward relationship context, risk profiles, trust-building conversations.
- A lawyer needs precedent recall, case history, what worked in court before and why.
- Your memory should look like no one else's. That's the core insight of this system.

Memory is an Infinite Game

Not a one-time setup. An evolving system.

Memory is an Infinite Game



Not a one-time setup. An evolving system.

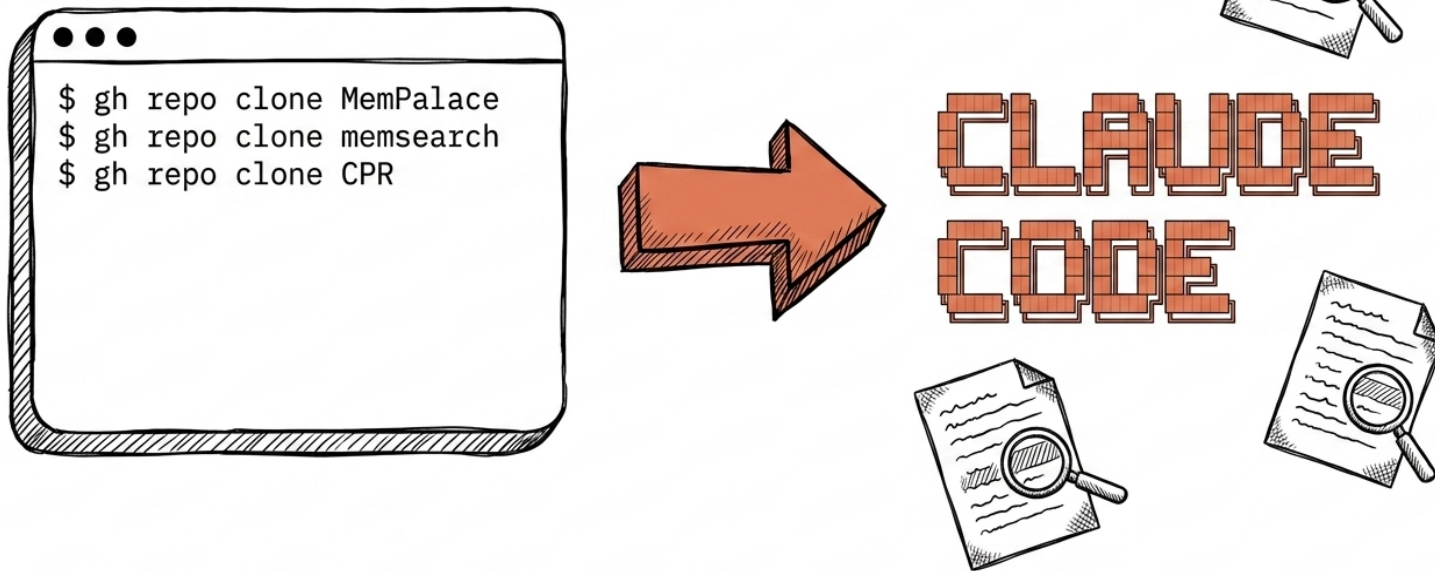
New paradigms ship constantly. Your system adapts.

- Six months ago nobody talked about multi-signal retrieval or salience scoring.
- Six months from now there will be paradigms that don't exist yet.
- Native Claude Code memory will keep improving. Any system you build needs to be designed for change.
- That's why this kit gives you a technique and an advisor, not just a one-time installer.

Clone and Audit

The technique: use Claude Code to explore open source repos.

Clone. Audit. Extract.



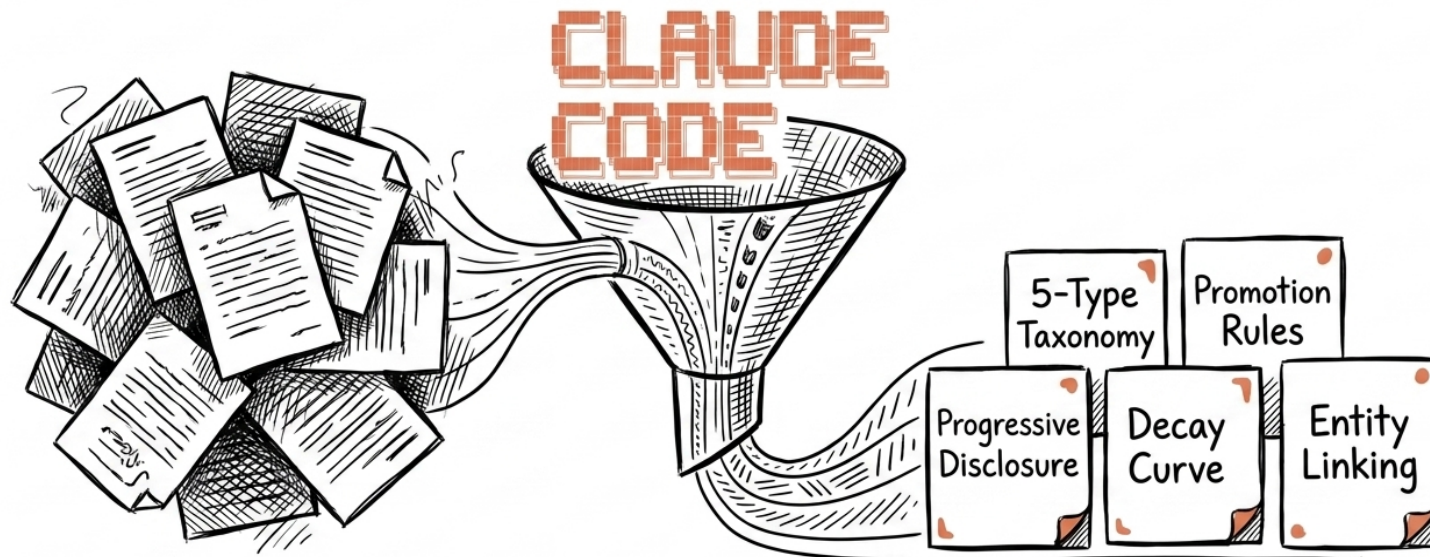
TERMINAL: Clone repos, then let Claude Code audit them

- Step 1: Clone repos that represent different memory philosophies (gh repo clone).
- Step 2: Open Claude Code and ask it to read the READMEs and extract design patterns.
- Step 3: Compare the tradeoffs across repos. What does each one do well?
- This technique works for ANY future repos. As new ones ship, you clone, audit, and cherry-pick.

Extract What Matters

Claude Code reads 35 repos. You keep the 5 patterns that fit.

Extract What Matters



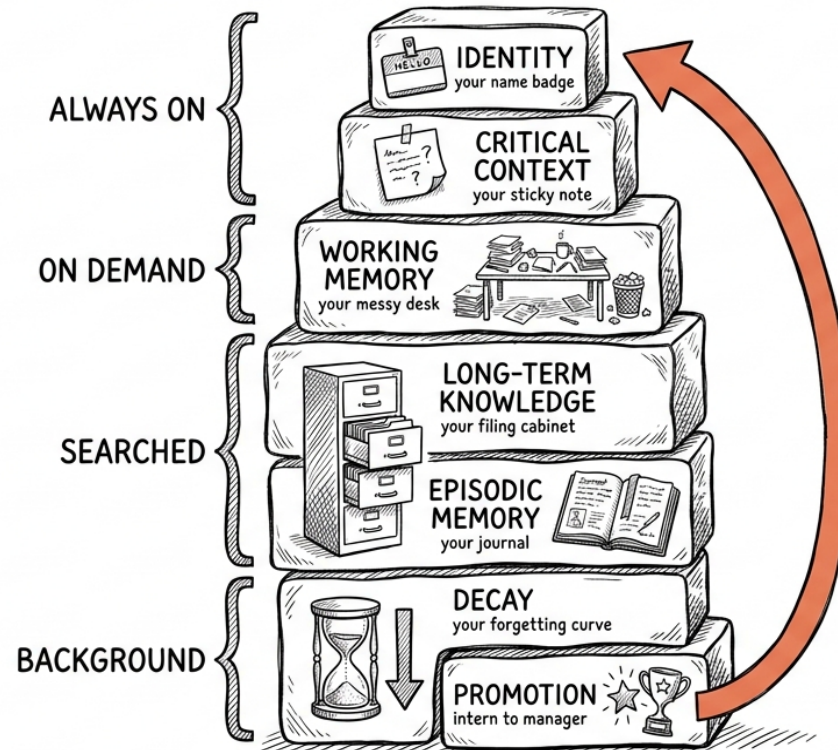
TERMINAL: Claude Code reads repos, you keep the patterns that fit

- The cherry-picking concept: take the taxonomy from one repo, the storage approach from another.
- Claude Code acts as your research assistant, turning overwhelming repos into clean pattern cards.
- Each pattern card represents a design choice: 5-type taxonomy, progressive disclosure, decay curve, promotion rules, entity linking.
- You assemble YOUR system from the patterns that match how YOU think and work.

Your Memory Stack

7 layers. Pick the ones you need. Most people need 3-4.

Your Memory Stack



- IDENTITY (~100 tokens): Your name badge. Who you are. Always loaded. Never changes.
- CRITICAL CONTEXT (~300 tokens): Your sticky note. Current project, blockers. Survives compaction.
- WORKING MEMORY (session-scoped): Your messy desk. Resets when you close.
- LONG-TERM KNOWLEDGE (searched): Your filing cabinet. Facts, decisions, patterns.
- EPISODIC MEMORY (archived): Your journal. Full conversation history. Preserves the WHY.
- DECAY (background): Your forgetting curve. Old memories compress. Frequently used ones resist.

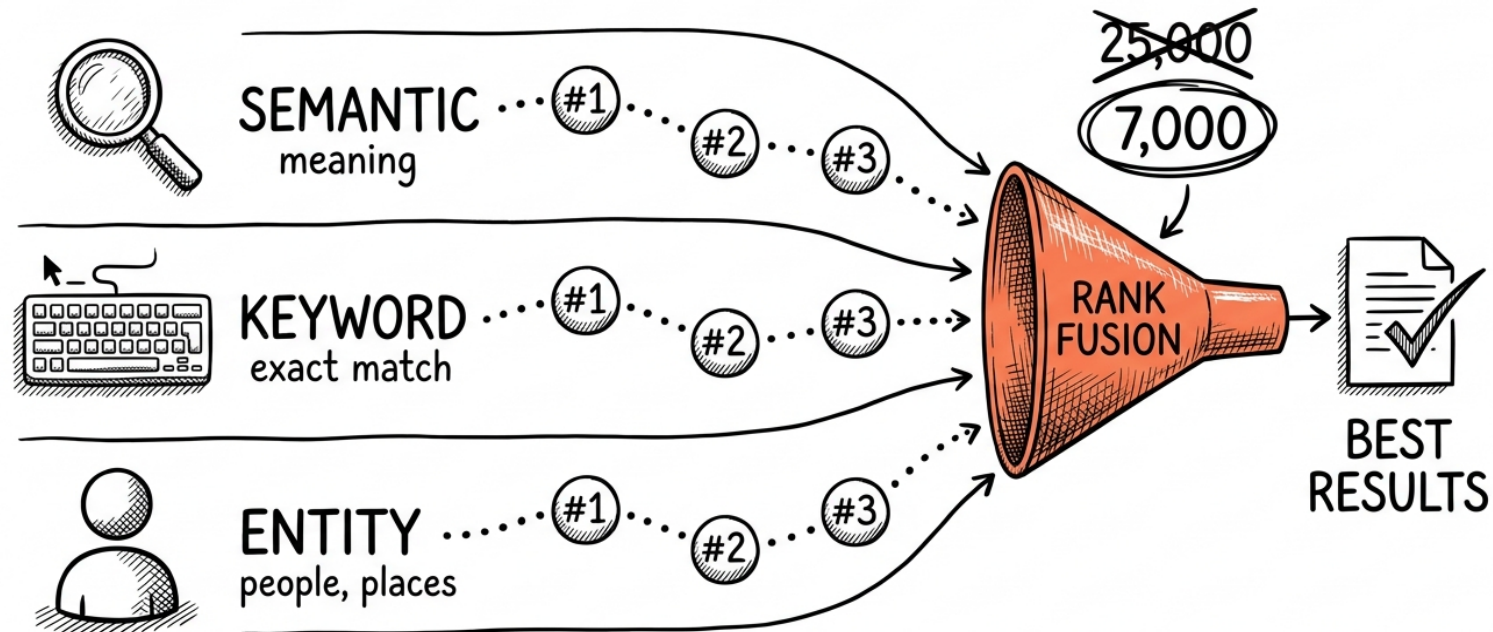
- PROMOTION (background): Intern to manager. Patterns seen 3x become permanent rules.

Early AI Dopters | <https://www.skool.com/earlyaidopters/about>

Multi-Signal Retrieval

Three signals. One answer. 70% fewer tokens.

Multi-Signal Retrieval



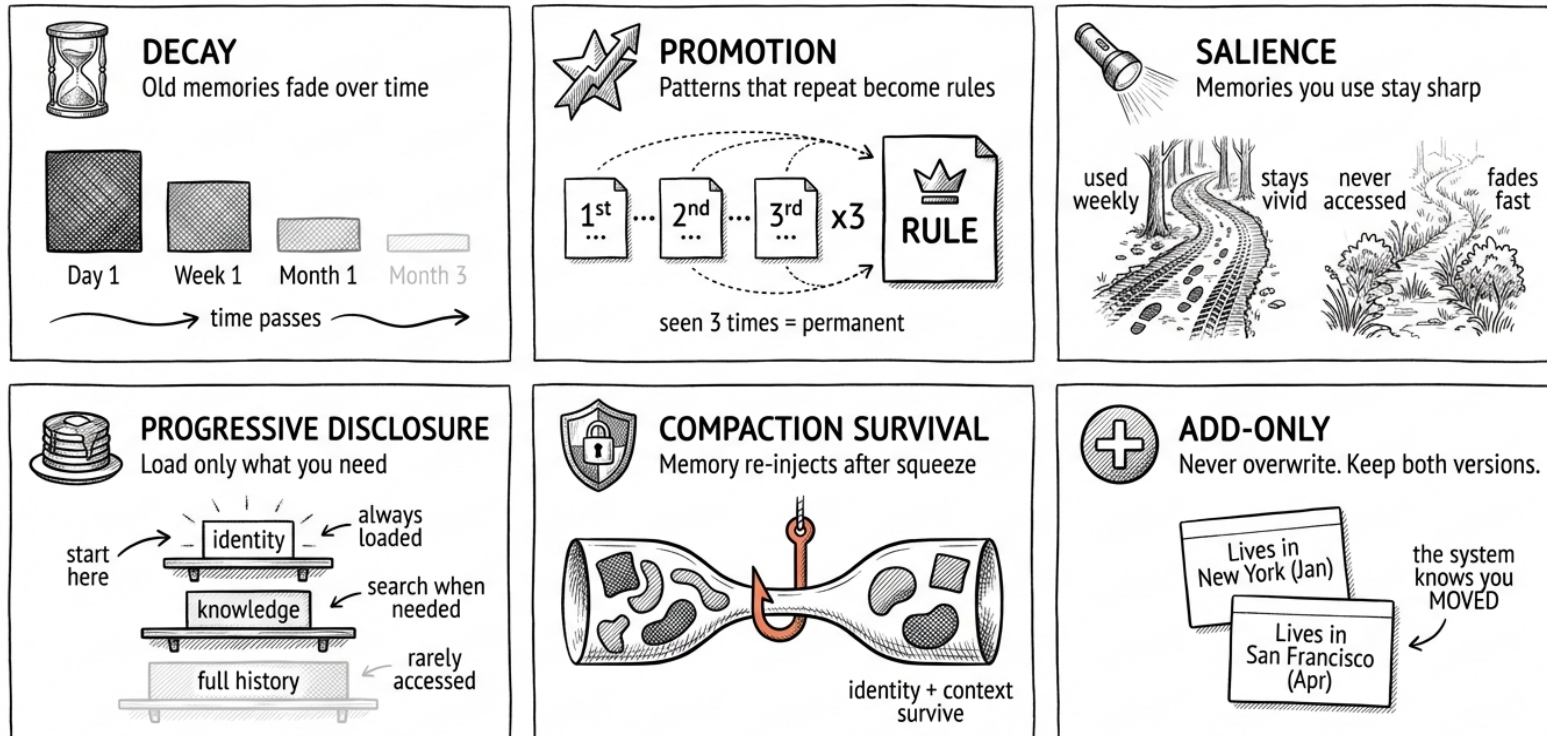
Three signals. One answer. 70% fewer tokens.

- SEMANTIC: finds by meaning. Search 'logins', find 'authentication.'
- KEYWORD: exact matches. Fast, simple, reliable.
- ENTITY: connects dots. Supabase mentioned in 50 conversations, all linked.
- Combine via Rank Fusion: ~7,000 tokens vs 25,000 for brute force. Same accuracy.

Memory Concepts Explained

The building blocks. You don't need all of them.

MEMORY CONCEPTS EXPLAINED

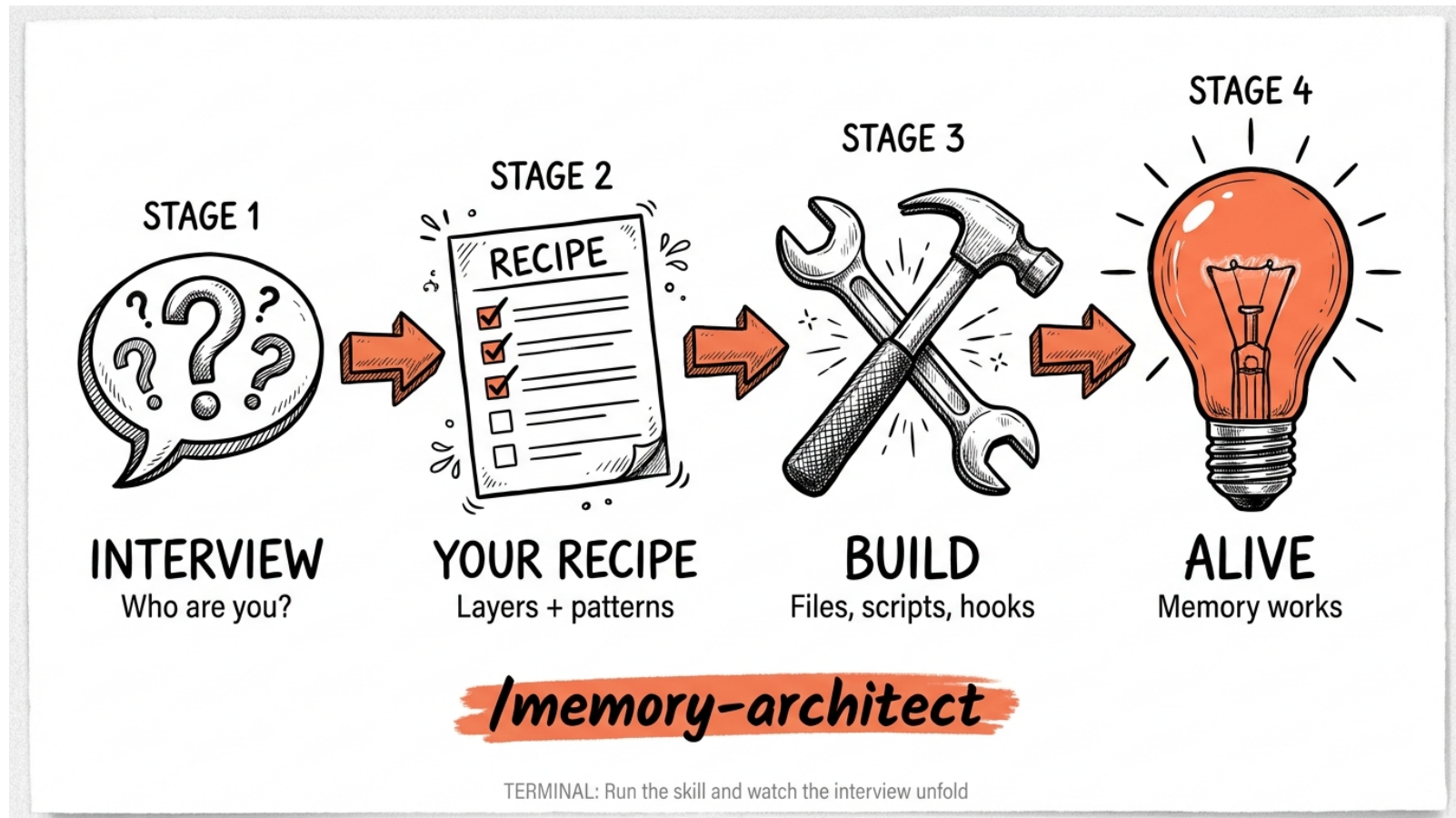


THE BUILDING BLOCKS. YOU DON'T NEED ALL OF THEM. PICK WHAT FITS.

- DECAY: Old memories fade. Day 1 = full detail. Month 3 = one-liner. Like your brain.
- PROMOTION: Patterns seen 3x become permanent rules. The observation is deleted. The rule stays.
- SALIENCE: Two paths in a forest. The one you walk daily stays clear. The unused one gets overgrown. Memories you USE resist decay.
- PROGRESSIVE DISCLOSURE: Carry a library card, not the whole library. Load identity always, search knowledge when needed.
- COMPACTION SURVIVAL: A hook re-injects your core memory right before Claude compresses. Identity and context survive.
- ADD-ONLY: Never overwrite. NY in January, SF in April. Both exist. The system knows you moved.

The /memory-architect Skill

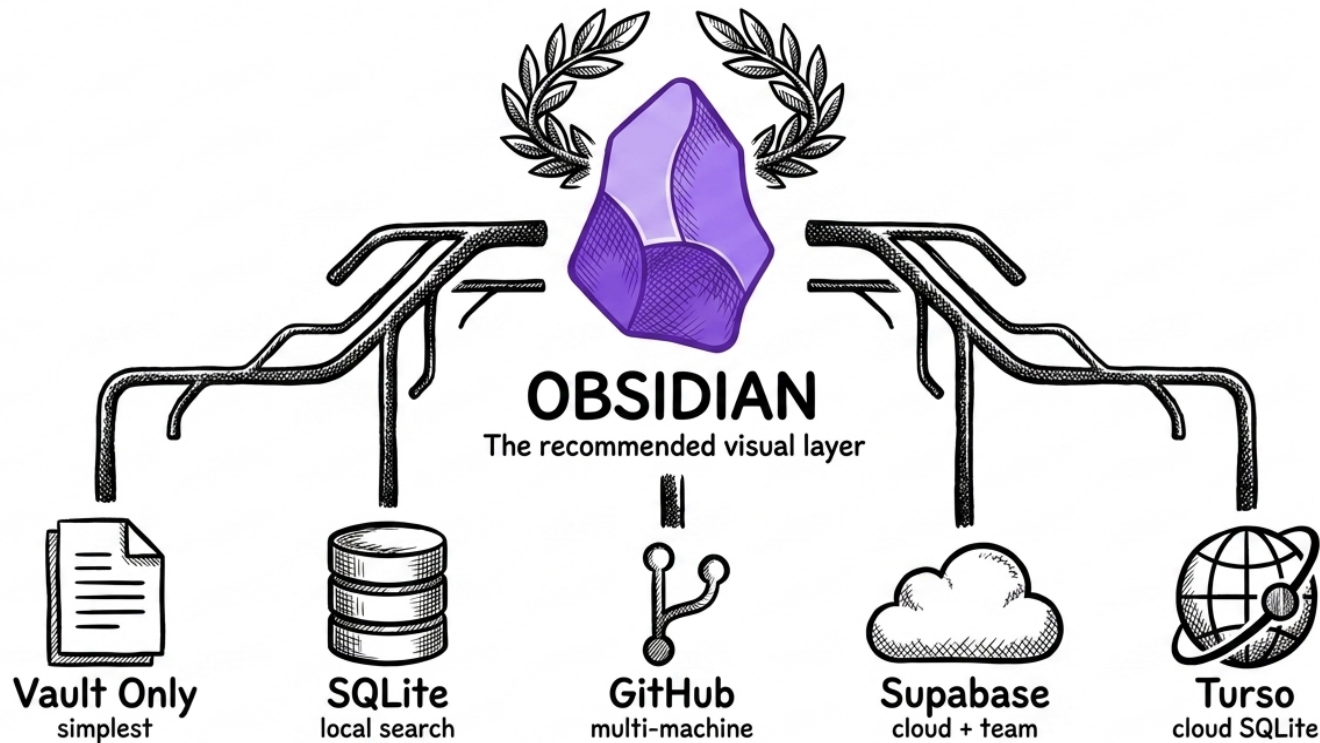
Interview, Recipe, Build, Alive.



- INTERVIEW: Asks who you are, teaches memory layers with visual diagrams, walks through 6 design decisions.
- RECIPE: Shows your personalized layer architecture, patterns, and infrastructure choice.
- BUILD: Creates the full memory structure pre-filled with your interview answers.
- ALIVE: Generates a handoff prompt for hook wiring. Memory works from the next session.
- Come back anytime in ADVISOR MODE to add layers, tune decay, switch backends.

Choose Your Platform

Obsidian is the recommended visual layer. Backend underneath.



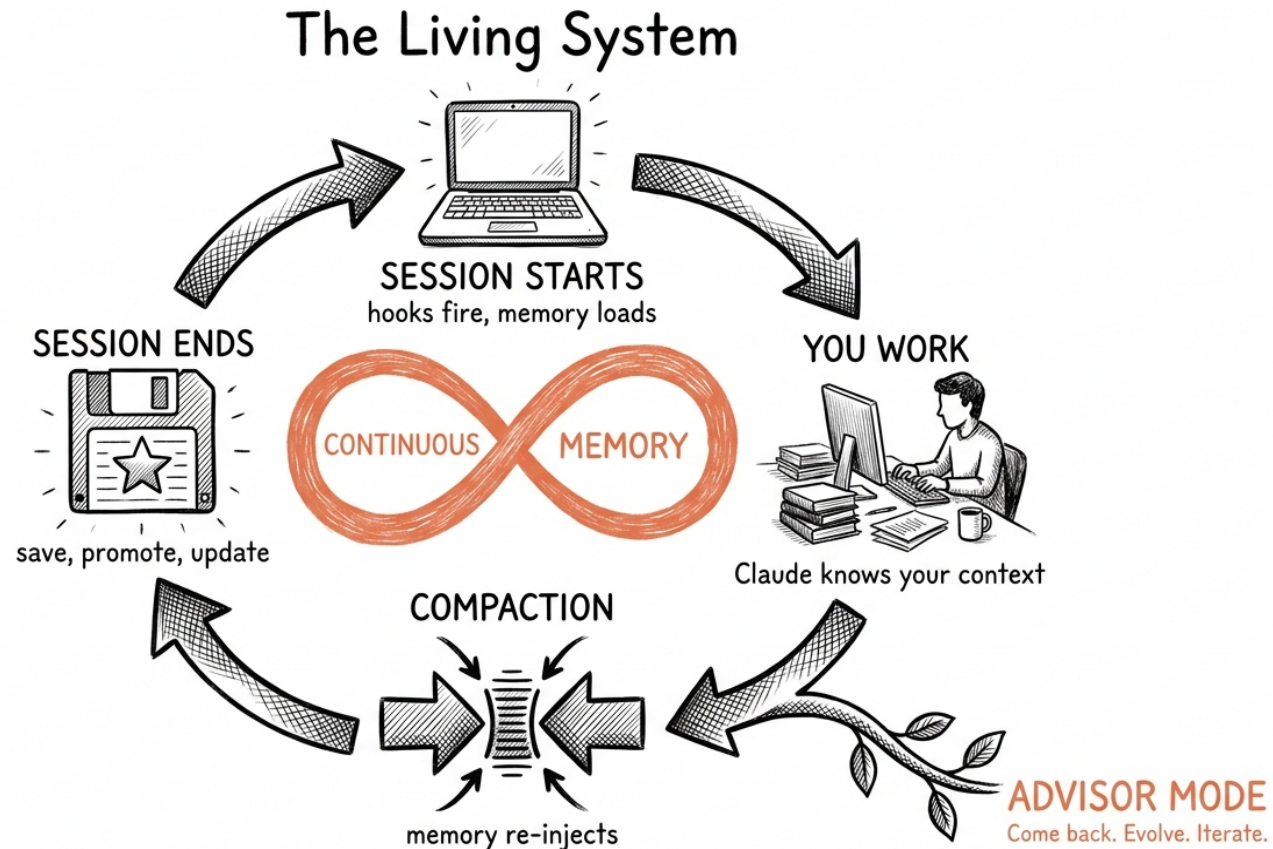
Path of Least Resistance

Obsidian is the interface. The backend is your choice.

- OBSIDIAN: Two doors to the same memory. You browse in the app. Claude reads via CLI.
- Vault only: simplest. SQLite: local semantic search. GitHub: multi-machine sync.
- Supabase: cloud + team access. Turso: SQLite in the cloud.
- Don't want Obsidian? Plain markdown works. Want free? SQLite. No API keys, no cloud.

The Living System

Continuous memory. Not a one-time install.



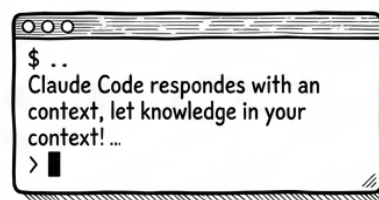
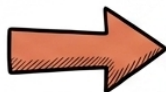
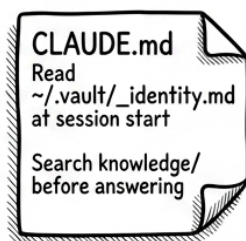
- SessionStart: loads identity + context. Claude knows you before you type.
- PreCompact: re-injects when context compresses. You never lose your place.
- SessionEnd: saves, promotes, updates context for next time.
- ADVISOR MODE: Run /memory-architect again anytime. Add layers. Tune decay. Evolve.

How Memory Gets Injected

Three approaches from simple to advanced.

How Memory Gets Injected

ROW 1.
Via CLAUDE.md



CLAUDE
CODE

Always in context. Claude reads it every session.

ROW 2.
Via Obsidian CLI
+ Hooks



SessionStart
obsidian read
"_identity.md"



PreCompact
obsidian read
"_context.md"

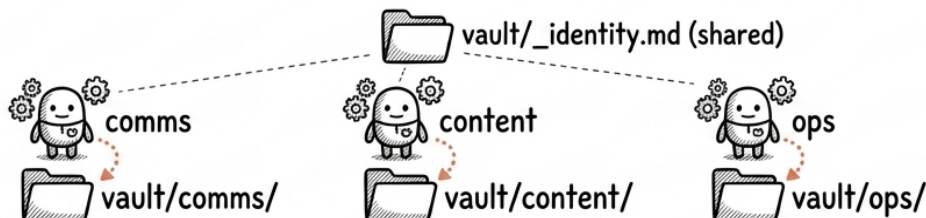


SessionEnd
obsidian write
"_context.md"



Automatic. Hooks fire at the right moments.

ROW 3.
Via Agent Scoping
(ClaudeClaw)



Each agent reads its own slice. Identity is shared.

Three ways. All automatic. All tangible.

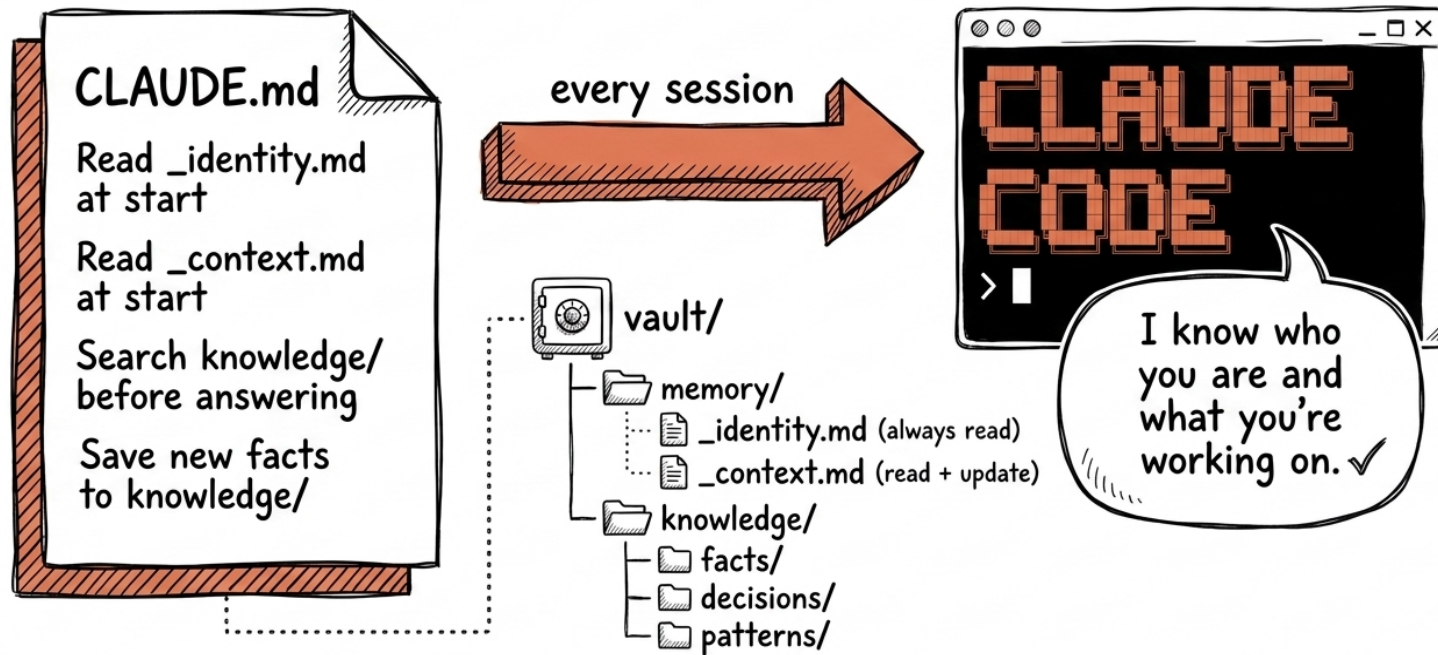
- Approach 1 (CLAUDE.md): Add instructions. Claude reads them every session. No hooks needed.
- Approach 2 (CLI Hooks): SessionStart, PreCompact, SessionEnd fire automatically.
- Approach 3 (Agent Scoping): Each agent reads its own vault folder. Identity shared.
- Recommended: combine hooks (inject data) + CLAUDE.md (teach behavior). Belt and suspenders.

Approach 1: CLAUDE.md

Simplest. No hooks. No CLI. Just instructions.

APPROACH 1: CLAUDE.md Injection

Simplest. No hooks. No CLI. Just instructions.



Claude reads **CLAUDE.md** every session. Your memory instructions are always present.

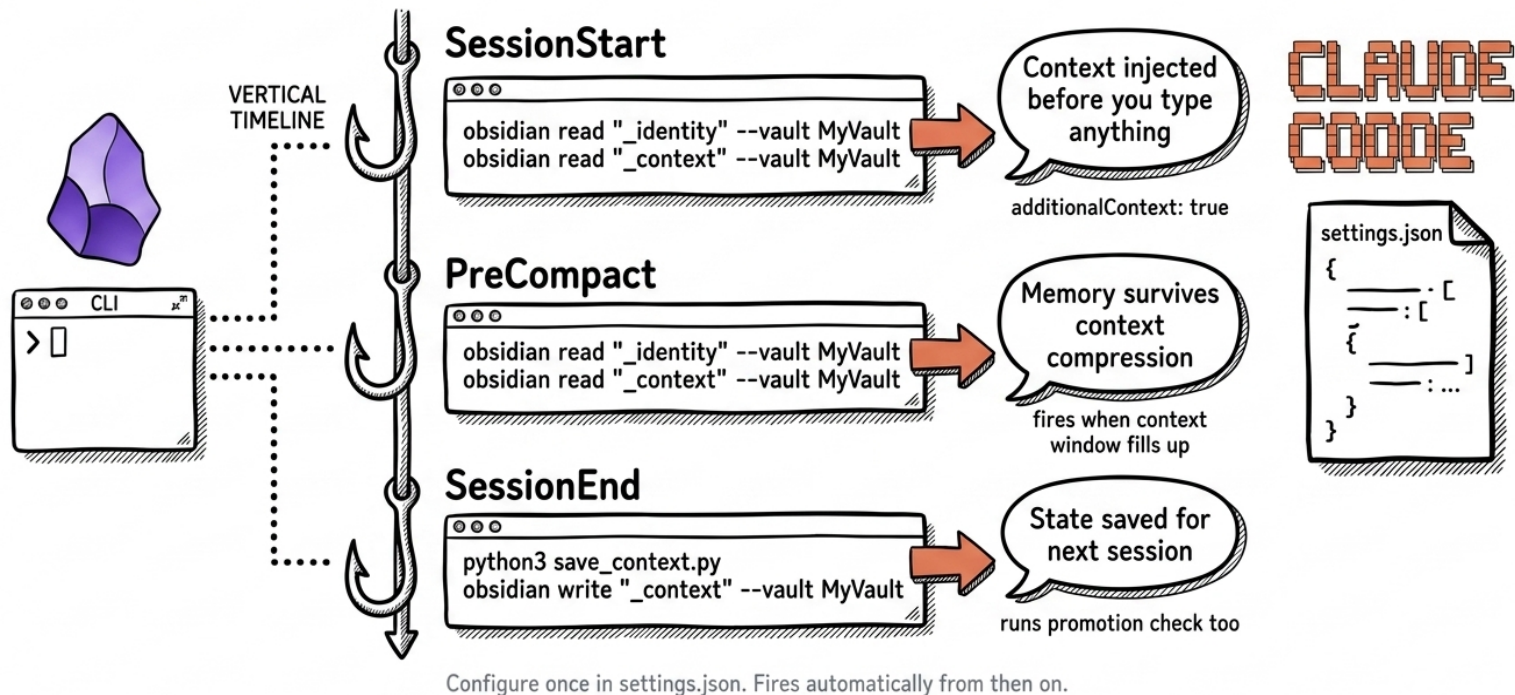
- Add a few lines to your CLAUDE.md: 'Read identity.md at start. Search knowledge/ before answering.'
- Claude reads CLAUDE.md every session. Your memory instructions are always present.
- No configuration files. No dependencies. Works immediately.
- This is where most people should start.

Approach 2: Obsidian CLI + Hooks

Automatic. Configure once. Fires every session.

APPROACH 2: Obsidian CLI + Hooks

Automatic. Hooks fire at the right moments. You do nothing.



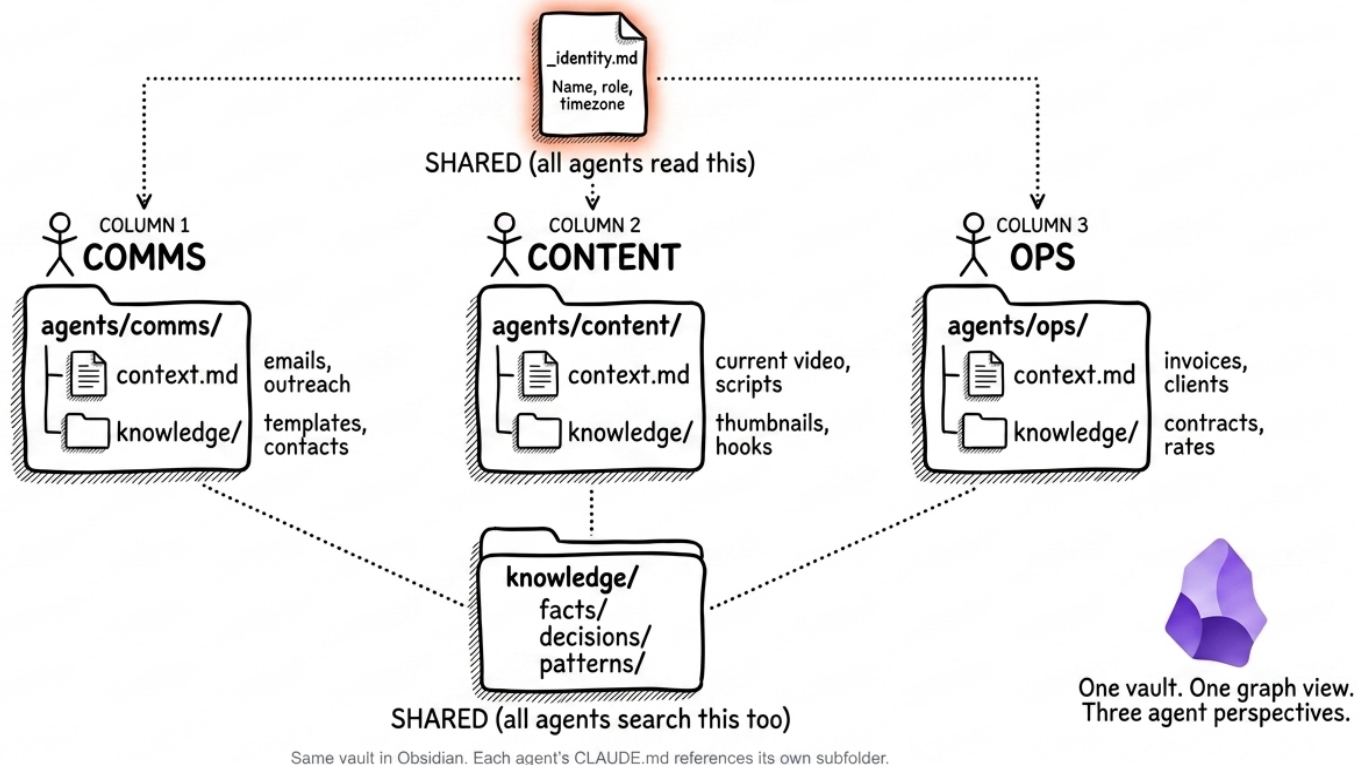
- SessionStart hook: reads identity.md and context.md, injects as hard context.
- PreCompact hook: re-injects when context window fills up. Memory survives compression.
- SessionEnd hook: saves updated context back to vault. Runs promotion check.
- The difference from Approach 1: hard context (guaranteed) vs soft instructions (behavioral).

Approach 3: Agent-Scoped Memory

Each agent reads its own slice. Identity shared.

APPROACH 3: Agent-Scoped Memory

Each agent reads its own slice. Identity is shared.



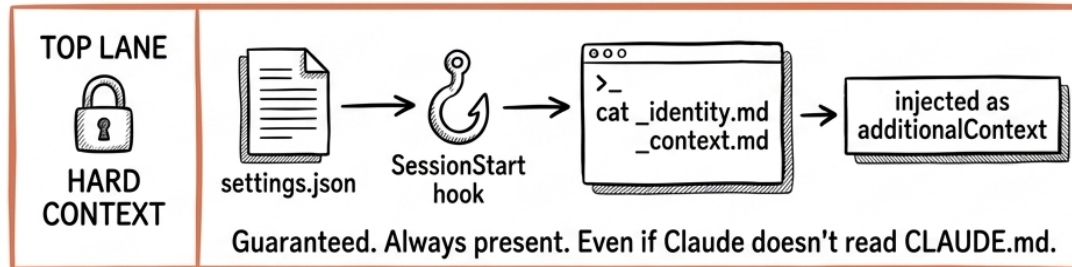
- Comms agent reads vault/agents/comms/. Content reads vault/agents/content/. Ops reads vault/agents/ops/.
- All agents share `_identity.md` (who you are) and the common `knowledge/` folder.
- One vault. One graph view in Obsidian. Each agent sees its own perspective.
- For multi-agent setups like ClaudeClaw.

Belt and Suspenders

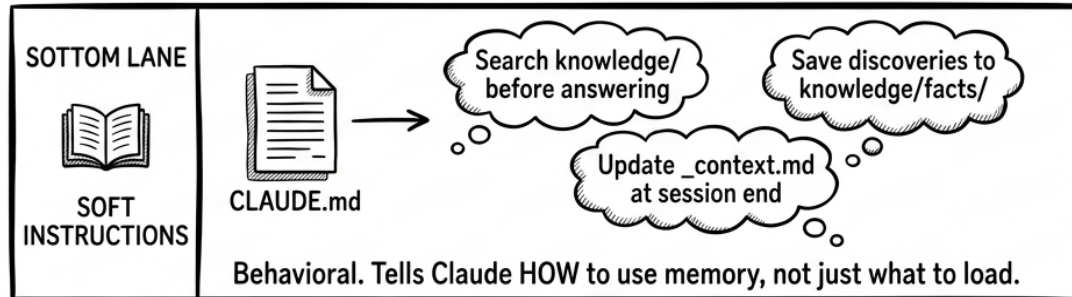
Hooks inject data. CLAUDE.md teaches behavior. Both layers.

BELT & SUSPENDERS: The Recommended Setup

Hooks inject. CLAUDE.md instructs. Both layers work together.



+ Hard context loads the data.
Soft instructions teach the behavior.



Two layers. If one fails, the other catches it.



Hook works + CLAUDE.md works = perfect	Hook fails + CLAUDE.md works = still has instructions, reads files itself
Hook works + CLAUDE.md fails = still has identity and context injected	Hook fails + CLAUDE.md fails = almost impossible

- Hooks = hard context. The data is guaranteed to be in Claude's context window.
- CLAUDE.md = behavioral instructions. Claude knows HOW to use the memory, when to search, when to save.
- If hooks fail, CLAUDE.md still has instructions. If CLAUDE.md fails, hooks still inject data.
- Once set up: open Claude Code, it knows who you are. Before you typed a single character.